

Spring Boot & Core Fundamentals

The Evolution of Backend Architecture:
From Tight Coupling to Auto-Configuration



Legacy Java

Tight Coupling



Spring Framework

Dependency Injection



Spring Boot 3

Auto-Configuration & Standalone

Based on the comprehensive guide by Hyder Abbas (TShaped Skills)

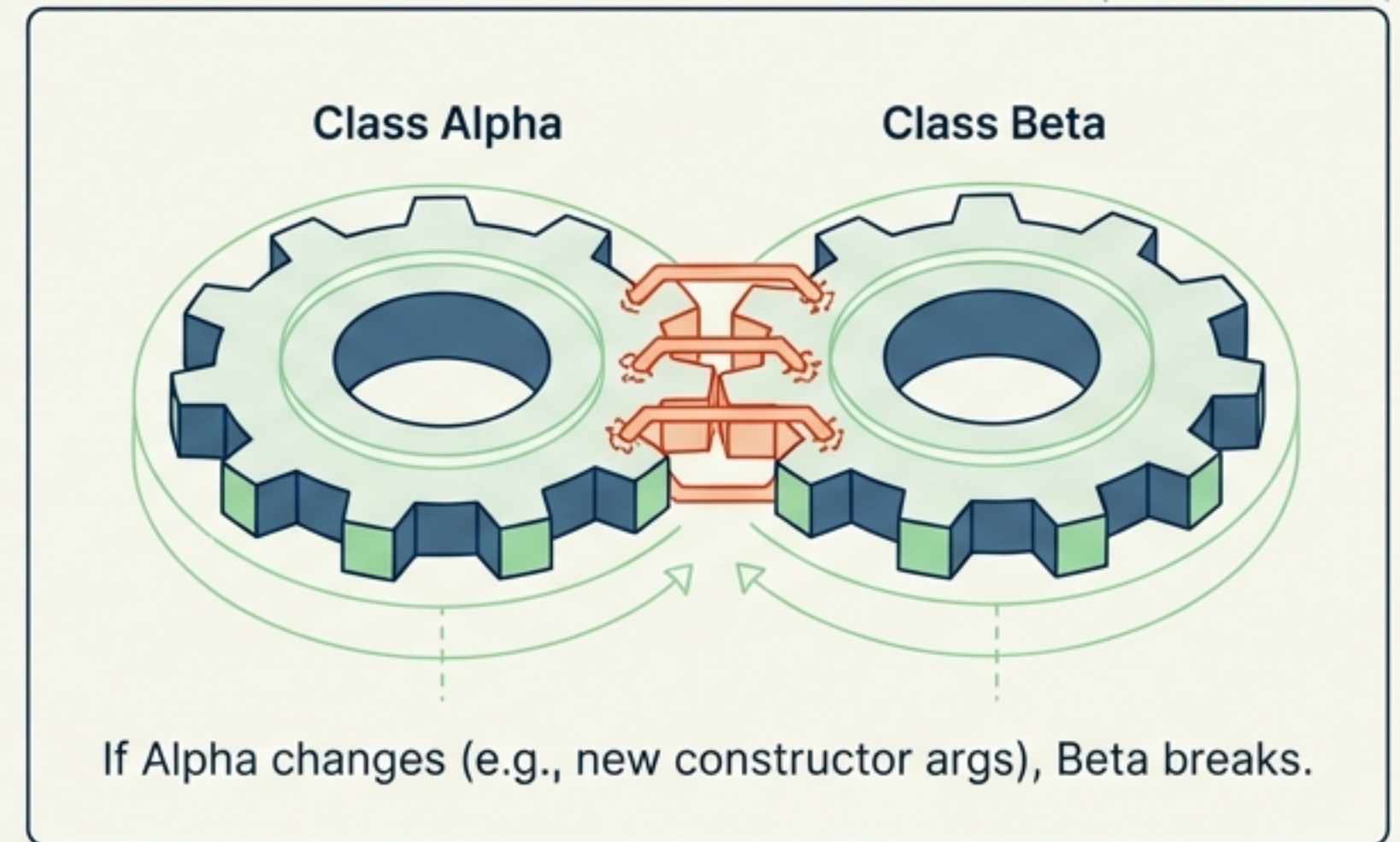
The Core Problem: The Fragility of Tight Coupling

The Code

```
1 public class Beta {  
2  
3     // Direct dependency creation  
4     Alpha a = new Alpha();  
5  
6     public void execute() {  
7         a.doWork();  
8     }  
9 }
```

Hard Dependency

The Consequence



- 1. **Hard to test** (cannot mock Alpha).
- ✓ 2. **Hard to maintain.**
- ✓ 3. **Changes ripple** through the codebase.

The Strategy: Inversion of Control (IoC)



Inversion of Control (IoC)

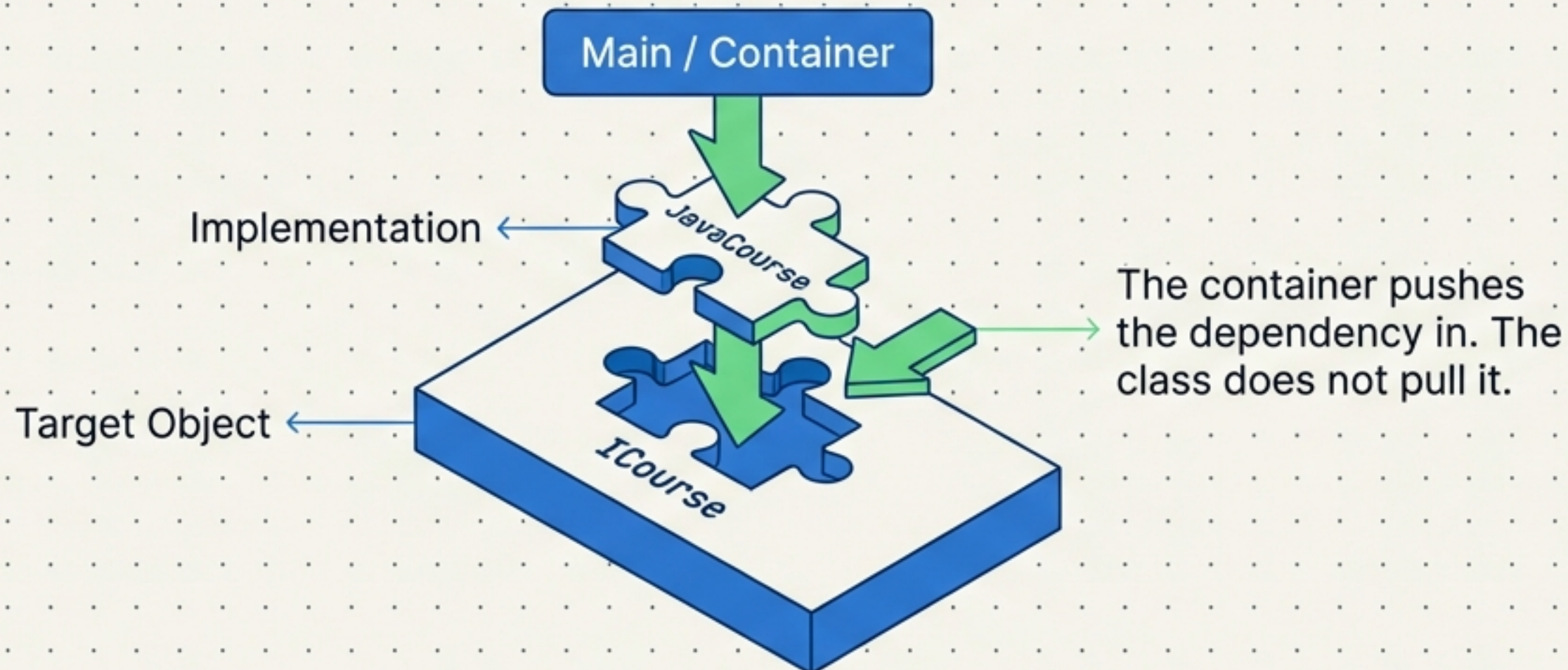
Transferring control of object creation from the developer to an external container.



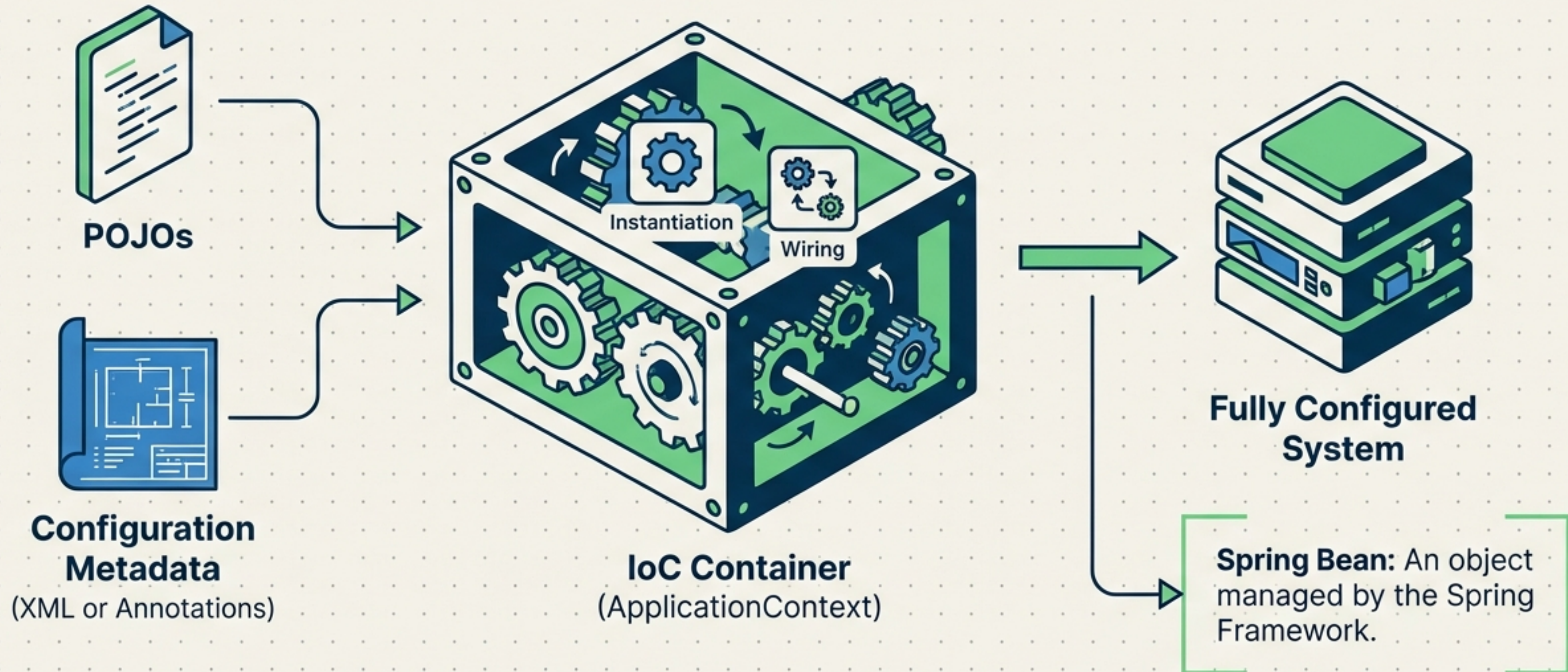
Dependency Injection (DI)

The technique where objects receive their dependencies from the outside.

Golden Rule: Code to Interface

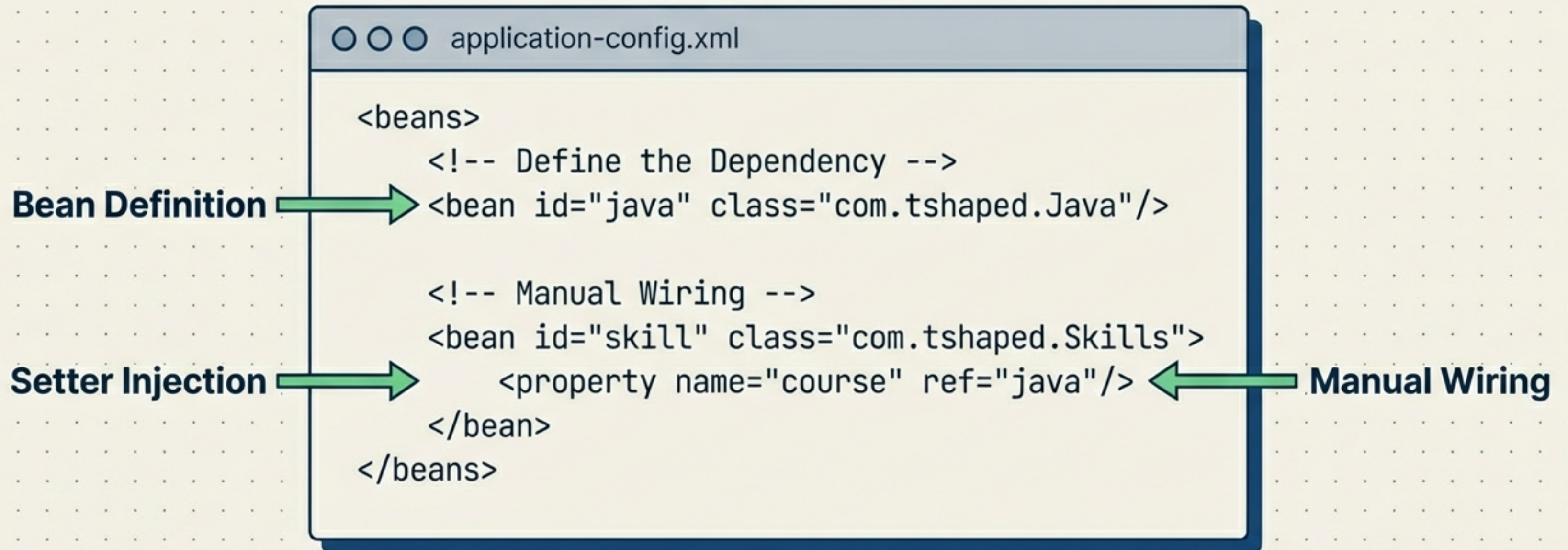


The Spring Container: Managing the Object Lifecycle



Evolution Phase 1: The XML Configuration Era

Separating Configuration from Code



Effective, but verbose. The logic is in Java, but the wiring is in XML.

Container Strategy: Eager vs. Lazy Initialization

BeanFactory (Legacy)



Lazy Initialization

Creates beans only when `getBean()` is called.

Pros/Cons:

- Saves memory.
- Configuration errors are caught late (at runtime).

ApplicationContext (Standard)



Eager Initialization

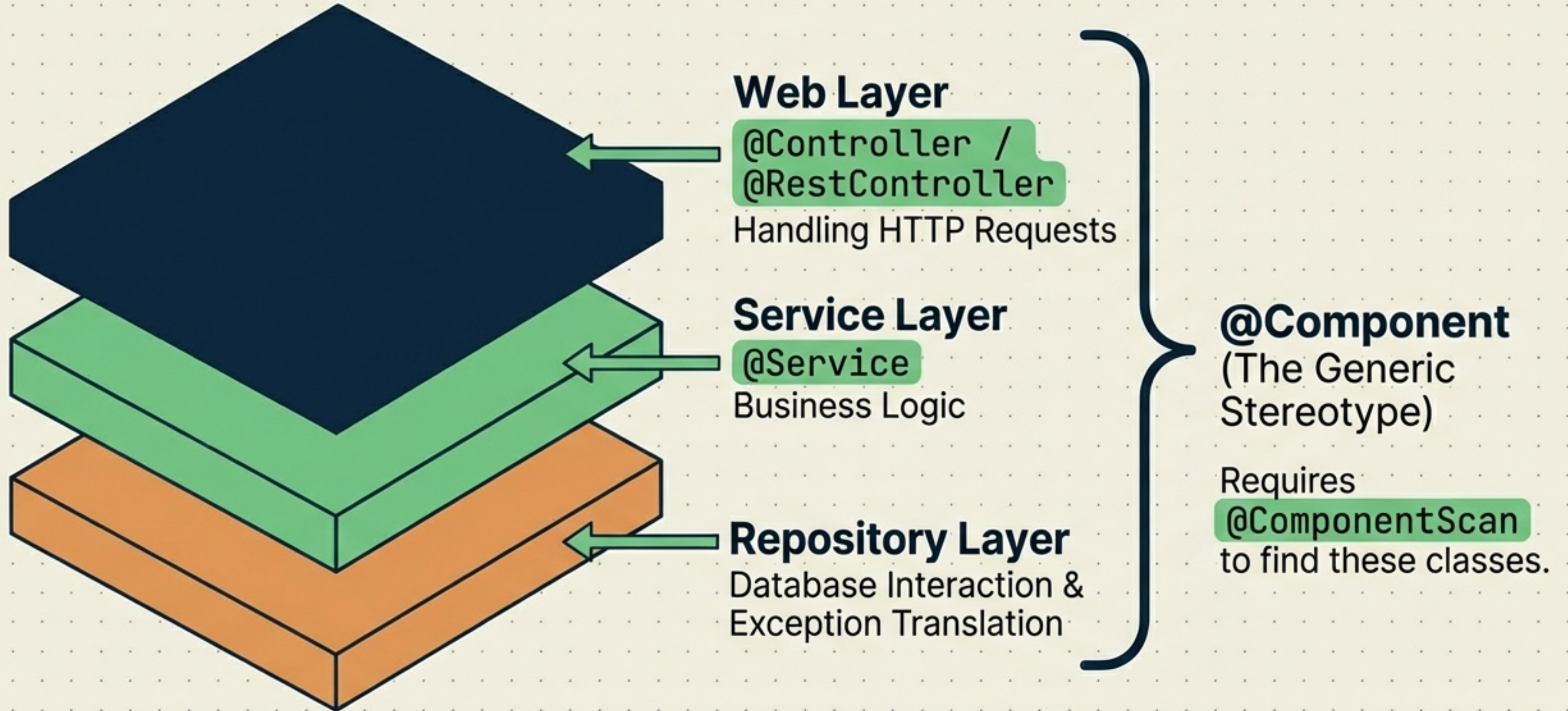
Creates all Singleton beans immediately at startup.

Pros/Cons:

- Fail-Fast strategy.
- Errors are caught immediately during deployment.

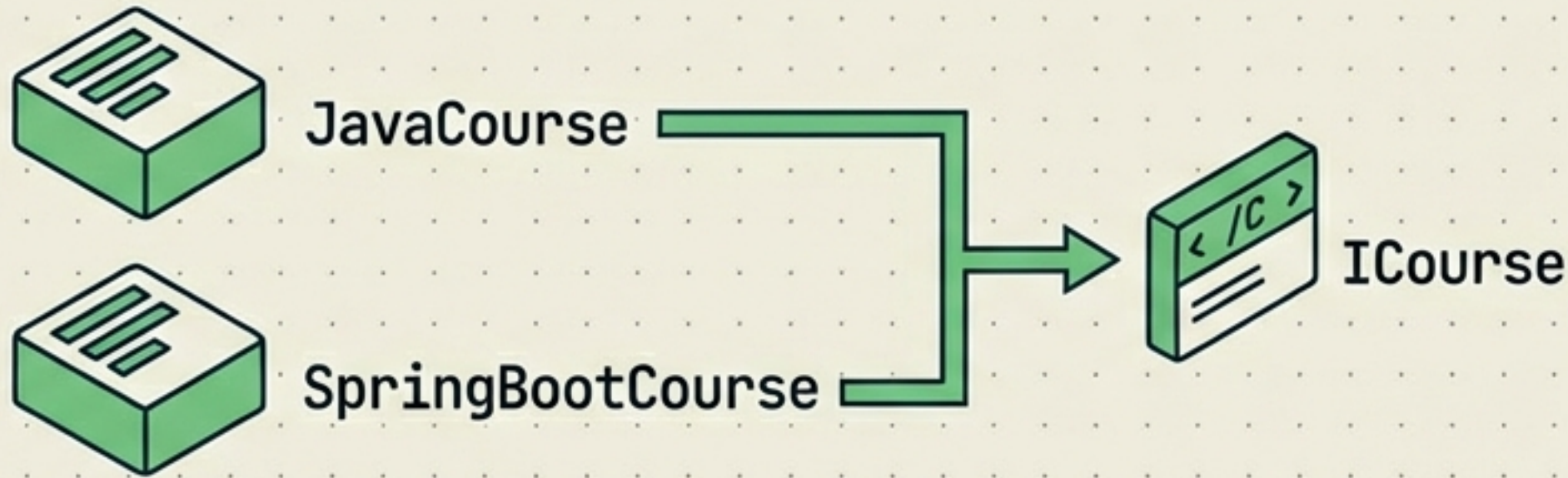
**Enterprise
Standard**

Evolution Phase 2: The Annotation Revolution



Handling Ambiguity in Auto-Wiring

Scenario: Controller asks for `ICourse`, but two Beans exist: `JavaCourse` and `SpringBootCourse`.



NoUniqueBeanDefinitionException: expected single matching bean but found 2

@Primary

The Default Choice.

```
@Primary  
public class JavaCourse...
```

@Qualifier

The Specific Override (Higher Priority).

```
@Autowired @Qualifier("springBoot")  
ICourse course;
```


Evolution Phase 3: Pure Java Configuration

Deleting the XML, keeping the type-safety.

Replaces
the XML file.

```
@Configuration
@ComponentScan(basePackages = "com.tshaped")
public class AppConfig {

    // Configuration logic in pure Java
}
```

Tells Spring
where to look
for
@Component
classes.

The bridge to modern Spring Boot. Refactor-friendly and type-safe.

Managing Third-Party Classes with @Bean

The Problem



Third-Party Libraries (e.g., `java.time.LocalDateTime`, `Gson`).

You cannot open the source code to add @Component.

The Solution



```
@Configuration
public class TimeConfig {

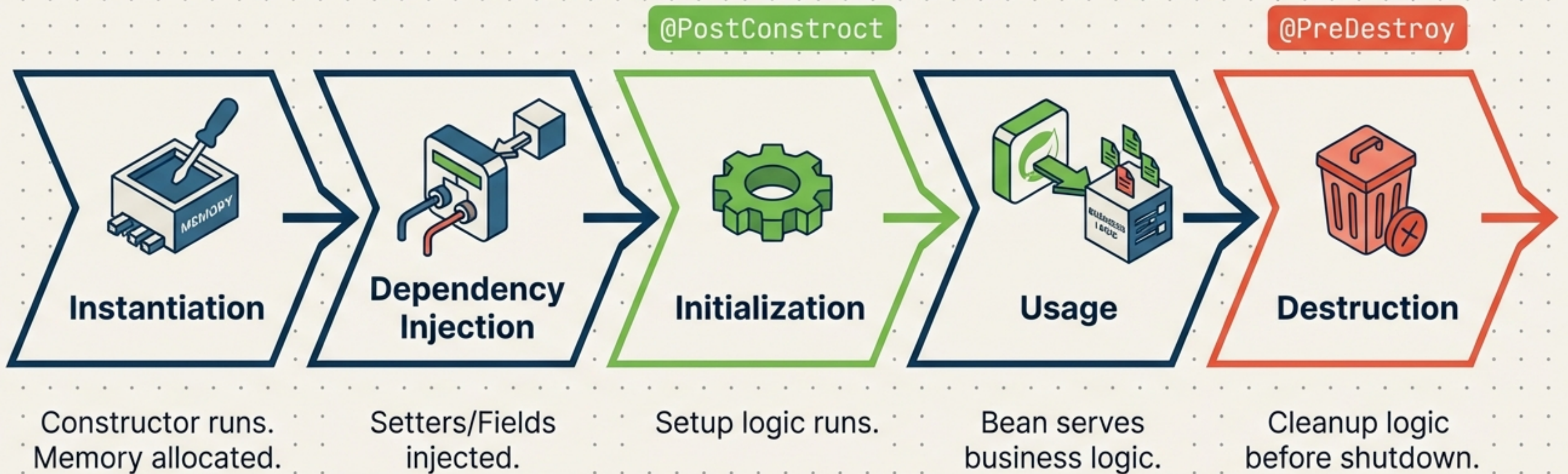
    @Bean
    public LocalDateTime createTime() {
        return LocalDateTime.now(); // Manual Creation
    }
}
```

Handed over to
Spring Context.

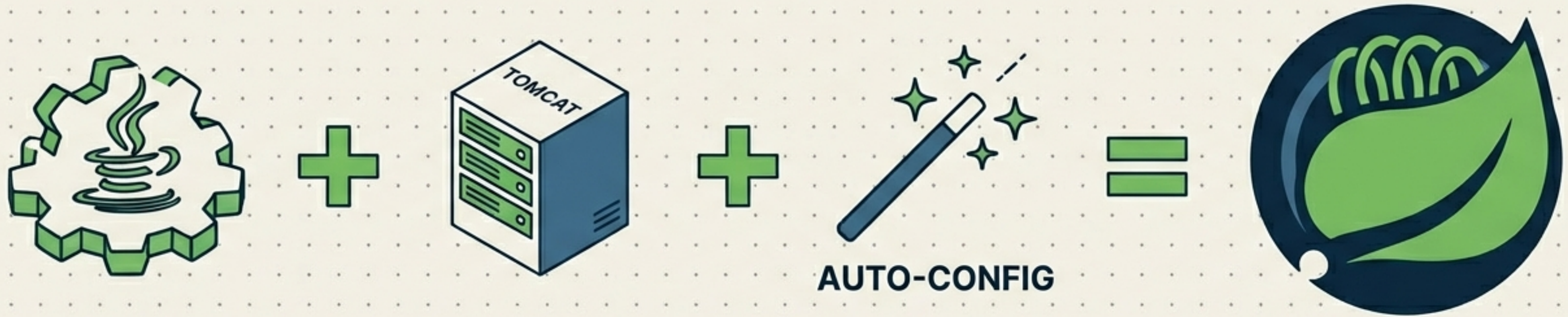


@Component is for your classes. **@Bean** is for external classes.

The Lifecycle of a Spring Bean



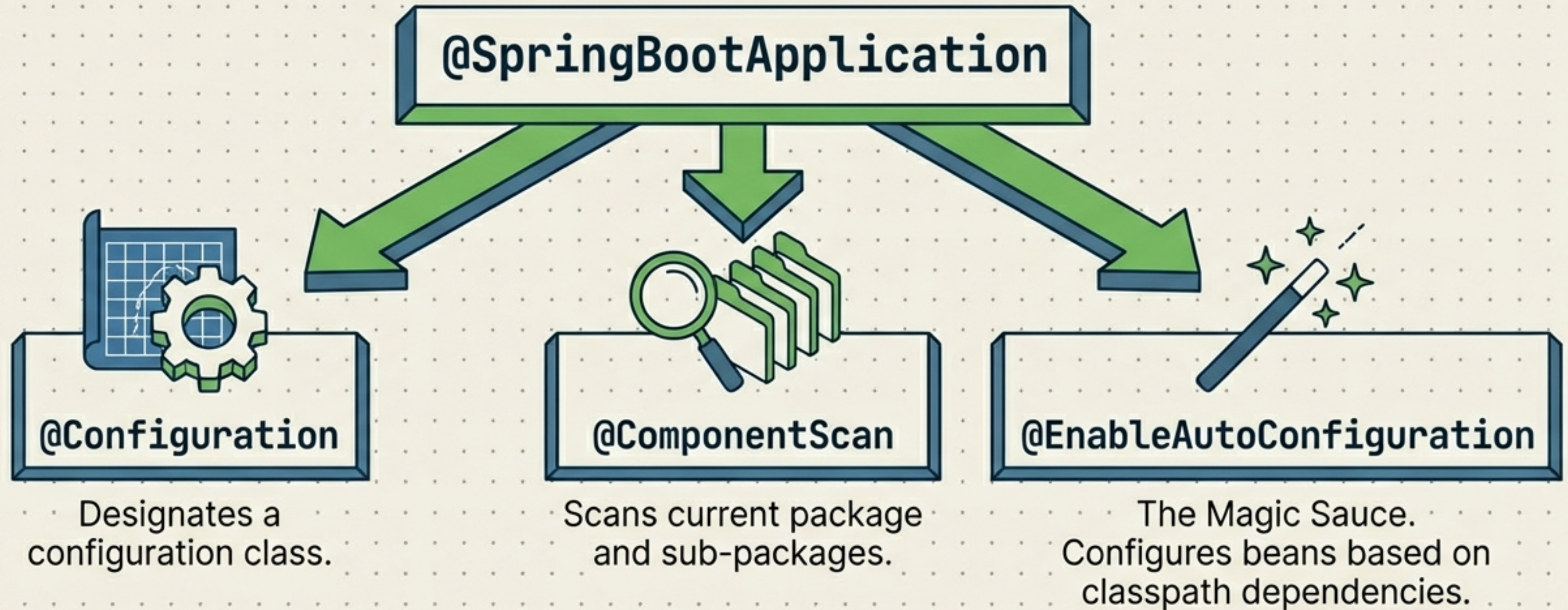
The Destination: Spring Boot



****Convention over Configuration****

Spring Framework	Spring Boot
Developer manages infrastructure (boilerplate).	Boot manages infrastructure. Developer focuses on Business Logic.

Inside the Magic: Auto-Configuration



Did you know? A simple 'Hello World' boots up ~52 default beans (Dispatchers, Error Handlers) automatically.

The Modern Spring Boot Workflow

`</>` Main.java

```
// 1. The Service ①
@Service
public class JavaCourse implements ICourse {
    public void study() { System.out.println("Learning..."); }
}

// 2. The Application & Injection ②
@SpringBootApplication
public class MyApp {
    @Autowired ICourse course; // 3. Injection ③

    public static void run(String[] args) {
        SpringApplication.run(MyApp.class, args); // 4. Execution ④
    }
}
```

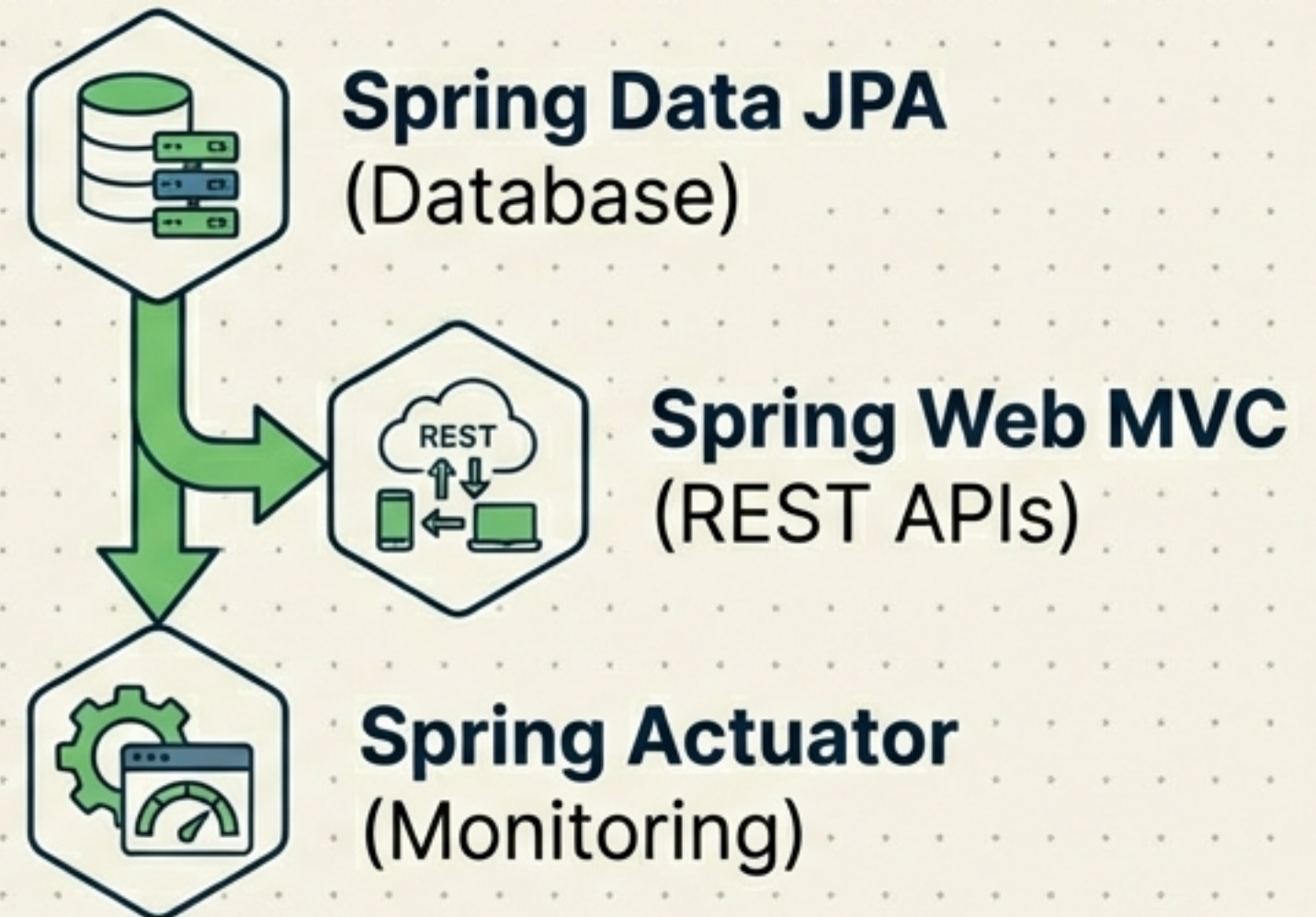
No XML. No manual context creation. Just Logic.

Key Takeaways & Next Steps

Summary Checkmarks

- ✓ **IoC & DI:** Decoupled dependencies for testable code.
- ✓ **Annotations:** Replaced XML, keeping config close to code.
- ✓ **Spring Boot:** Automated setup via Convention over Configuration.

The Ecosystem Map



Credits: Based on the tutorial by Hyder Abbas (TShaped Skills).